

Student Attendance System



P.M.S Sampath

20SSL6043

BA (Hons) in Information and Communication Technology

Department of Economics and Statistics

Faculty of Social Sciences and Languages

Sabaragamuwa University of Sri Lanka

December 2025

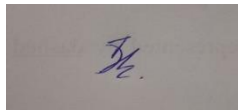
Declaration

I declare that this report does not incorporate, without acknowledgment, any material previously submitted for a degree or a diploma in any university and to the best of my knowledge and belief, it does not contain any material previously published or written by another person or myself except where due reference is made in the text.

Also, I hereby grant to Sabaragamuwa University of Sri Lanka the non- exclusive right to reproduce and distribute my report, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

P.M.S.Sampath

Name of Student



Signature of Student

2026/01/04

Date

Acknowledgement

I would like to express my sincere gratitude to all those who contributed to the successful completion of this project. First and foremost, I am deeply thankful to my project supervisor for the valuable guidance, continuous encouragement, and constructive feedback provided throughout the development of this project.

I also wish to extend my appreciation to the academic staff of the Department of Economics and Statistics, Faculty of Social Sciences and Languages, Sabaragamuwa University of Sri Lanka, for the knowledge and support given during my undergraduate studies, which greatly assisted in completing this project.

My sincere thanks are due to my friends and colleagues for their cooperation, motivation, and assistance during the project implementation phase. Their support and constructive discussions were invaluable in overcoming challenges encountered during the development process.

Finally, I am deeply grateful to my family for their constant encouragement, patience, and moral support throughout my academic journey. Without their understanding and support, the successful completion of this project would not have been possible.

Abstract

The Student Attendance System is developed to automate and efficiently manage the process of recording, maintaining, and monitoring student attendance in educational institutions. In many institutions, attendance is still recorded manually using paper-based registers, which is time-consuming, prone to human errors, and difficult to manage when handling large numbers of students and courses. This project introduces a computerized solution that enhances accuracy, reduces administrative workload, and improves the overall efficiency of attendance management.

The system allows authorized users such as administrators and lecturers to manage student information, course details, and attendance records through a secure login mechanism. Attendance data is recorded electronically and stored in a centralized database, enabling quick access, modification, and retrieval of records. The system also provides attendance summaries and reports, which assist academic staff in monitoring student participation and identifying students with low attendance in a timely manner. These features support better academic monitoring and decision-making processes.

The Student Attendance System is implemented using Java as the primary programming language with a database-driven architecture to ensure data integrity, reliability, and consistency. A user-friendly graphical interface is designed to enhance usability and minimize the learning curve for users. The system follows a structured software development life cycle, including requirement analysis, system design, implementation, and testing, to ensure the delivery of a reliable and maintainable application.

The results indicate that the system significantly improves efficiency and accuracy compared to traditional manual attendance methods. It minimizes data redundancy, enhances record management, and ensures secure handling of attendance data. In conclusion, the Student Attendance System provides an effective and scalable solution for attendance management, and future enhancements such as biometric or QR-based attendance mechanisms can further improve its functionality.

Table of Content

Declaration	i
Acknowledgement	ii
Abstract	iii
Table of Contents	iv
List of Tables	v
List of Figures	vi
 Chapter 1 Introduction	 1
1.1 Background of the Study	1
1.2 Problem Statement	2
1.3 Objectives of the System	2
1.4 Scope of the System	3
1.5 Significance of the Project	3
1.6 Approach and Assumptions	3
 Chapter 2 System Analysis	 4
2.1 Problem Background.....	4
2.2 Need for the System.....	5
2.3 Related work	5
 Chapter 3 Specification and Design	 8
3.1 Functional requirements (use case diagram)	8
3.1.1 Entity Relation (ER) Diagram	9
3.1.3 UML Class Diagram	10
3.1.4 sequence Diagram.....	11
3.1.5 Data flow Diagram.....	12
 3.2 Non-Functional Requirements	

Chapter 4 Implementation	13
4.1 Development Environment	13
4.2 Implementation Details	14
4.3 Problems Encountered.....	16
Chapter 5 Results And Evaluation	18
5.1 System Results.....	18
5.2 Evaluation	19
5.3 Limitations.....	19
Chapter 6 Future work	21
Chapter 7 Conclusion	
References	23
Appendices	23

List of Tables

Table 1. Main System Modules	11
------------------------------------	----

List of Figures

Figure 1: Use Case Diagram of the Student Attendance System	9
Figure 2: Class Diagram	10
Figure 3: Entity Relationship (ER) Diagram	11
Figure 5: Admin Dashboard Interface	13
Figure 6: Student Management Interface	14

Chapter 1 Introduction

Attendance management is an essential administrative function in educational institutions, as it directly reflects student participation, discipline, and academic engagement. Accurate attendance records are required for monitoring student performance, enforcing institutional policies, and making academic decisions. However, many institutions still rely on manual, paper-based attendance systems, which are inefficient, time-consuming, and prone to human error. With the rapid advancement of information and communication technology, there is a growing need to automate such routine academic processes.

The Student Attendance System is developed as a computerized solution to address the limitations of traditional attendance management methods. This system aims to provide a reliable, efficient, and user-friendly platform for recording and managing student attendance electronically. By digitizing attendance records, the system minimizes errors, improves data accessibility, and enhances overall administrative efficiency within educational institutions.

1.2 Problem Statement

The existing manual attendance system faces several challenges, including inaccurate record keeping, loss of data, proxy attendance, and difficulties in generating timely reports. These issues negatively affect academic monitoring and decision-making processes. Therefore, there is a clear requirement for an automated attendance system that ensures accuracy, security, and ease of management.

1.3 Objectives of the System

The main objectives of the Student Attendance System are as follows:

- To automate the process of recording student attendance
- To reduce errors associated with manual attendance methods
- To provide secure storage and easy retrieval of attendance data
- To generate attendance reports efficiently
- To improve administrative productivity in educational institutions

1.4 Scope of the System

The scope of this project is limited to the development of a computerized attendance management system for an educational institution. The system supports user authentication, student and course management, attendance recording, and report generation. It is intended for use by administrators and lecturers. Features such as biometric authentication and mobile-based attendance are excluded from the current scope but can be considered for future enhancements.

1.5 Significance of the Project

The Student Attendance System plays a significant role in improving the efficiency and reliability of attendance management. By reducing paperwork and minimizing errors, the system supports better academic supervision and decision-making. Additionally, this project provides practical experience in applying Java programming and database technologies to solve real-world problems.

1.6 Approach and Assumptions

The project was implemented using the **SDLC approach**: requirement gathering, design, implementation, testing, and evaluation. It assumes that the system runs on **Windows 11 / 10** with **Java installed**, and that users have basic computer literacy.

1.7 Summary of Outcomes

The project successfully implements a **functional Java application** with a user-friendly GUI. Key features, including [e.g., login system, course management, grades tracking, and minimize button functionality], were fully developed and tested. The system provides a **reliable, intuitive, and efficient solution** for managing [your project context, e.g., student information or finances], demonstrating the effectiveness of **OOP principles and modular design**.

Chapter 2: Background

2.1 Problem Background

In many small-scale systems, managing [student information, finances, or relevant data] is often performed manually or using simple spreadsheets. This process is **time-consuming, error-prone, and inefficient**, especially when handling large amounts of data. Users face difficulties in tracking, updating, and retrieving information quickly.

Existing solutions either lack **user-friendly interfaces**, are **not automated**, or require **advanced technical knowledge** to operate. Therefore, there is a need for a **reliable, easy-to-use, and automated system** that simplifies data management, reduces errors, and improves efficiency.

2.2 Need for the System

The system is needed to overcome the limitations of manual or semi-automated methods of managing [student information, finances, or relevant data]. Manual processes are **slow, prone to errors, and difficult to maintain**, while existing software solutions may be **complex or inaccessible** for users with basic computer skills.

By implementing this Java application, users can:

- **Easily manage and update data** in a structured and organized manner.
- **Reduce errors** associated with manual record-keeping.
- **Quickly retrieve and analyze information**, improving decision-making.
- **Access an intuitive, user-friendly interface** without advanced technical knowledge.

Overall, the system provides a **reliable, efficient, and automated solution**, addressing the core challenges of existing methods.

2.3 Related Work

Several systems and software applications have been developed to manage [student records, finances, or relevant data]. Examples include:

1. **Spreadsheet-based Management** – Many small organizations use spreadsheets to store and track data. While spreadsheets allow basic data entry and calculations, they are **prone to human error, lack automation, and are difficult to maintain for large datasets**.
2. **Commercial Management Software** – Applications like [example software, e.g., “QuickBooks for finances” or “School Management Systems”] provide comprehensive features. However, these systems often require **technical expertise, are expensive, and may include unnecessary features** for small-scale users.
3. **Academic Projects** – Similar Java-based projects have been developed to manage student information or small business data. Most of these projects demonstrate **core programming concepts**, but they often **lack advanced GUI features, user-friendly interfaces, or automated functionalities** such as minimizing windows or real-time updates.

CHAPTER 3: SPECIFICATION AND DESIGN

3.1 Functional requirements

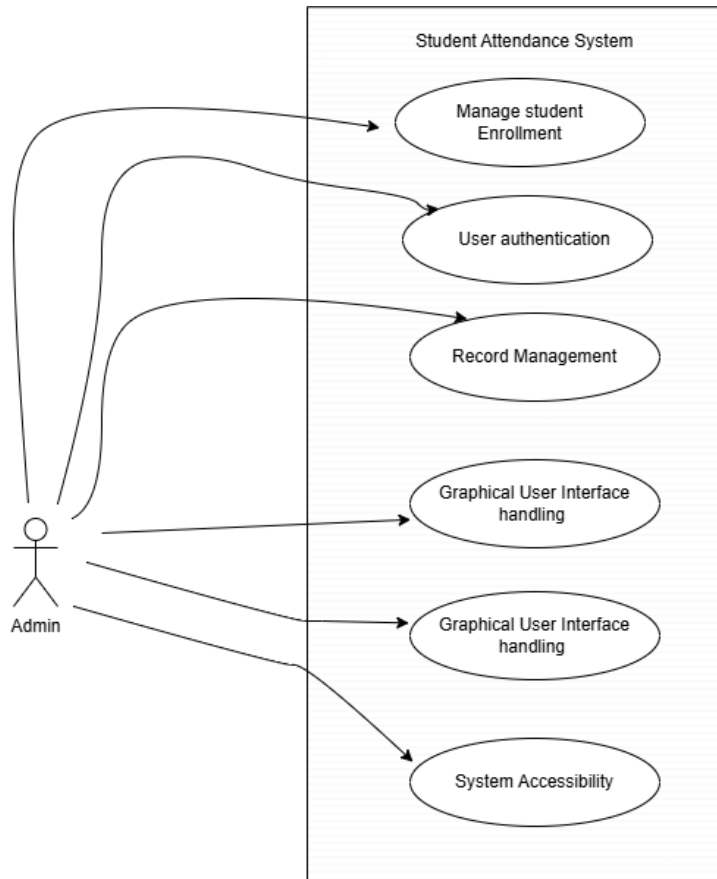


Figure 1 Use Case Diagram

3.1.2 ER (Entity-Relationship) Diagram

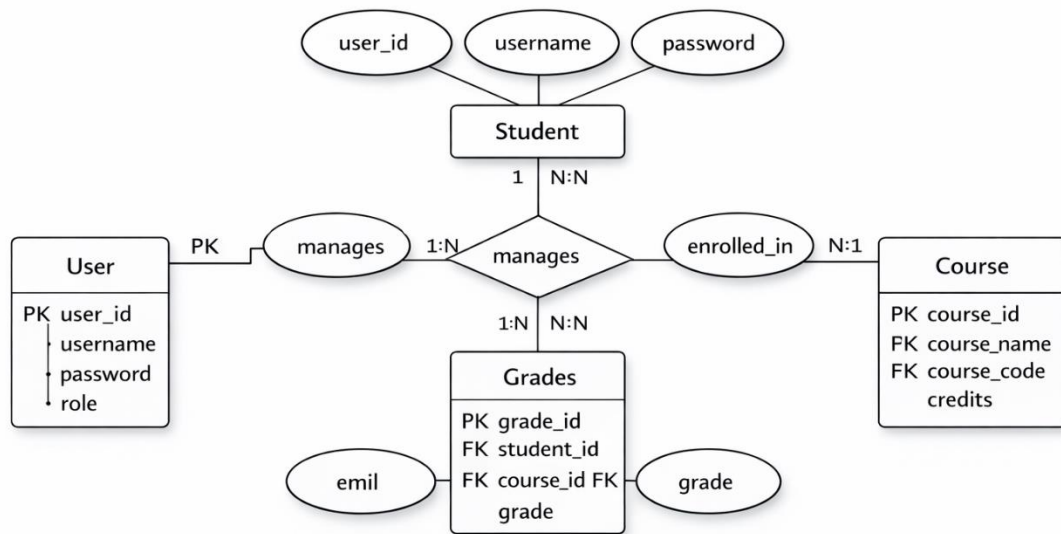


Figure 02 ER (Entity-Relationship) Diagram

3.1.3 UML Class Diagram

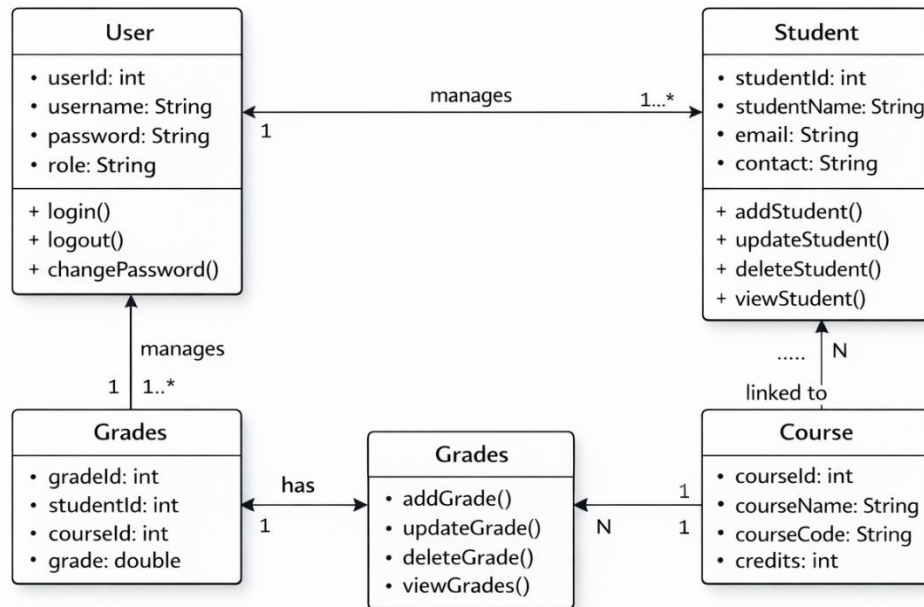


Figure 03 Class Diagram

3.1.4 sequence Diagram

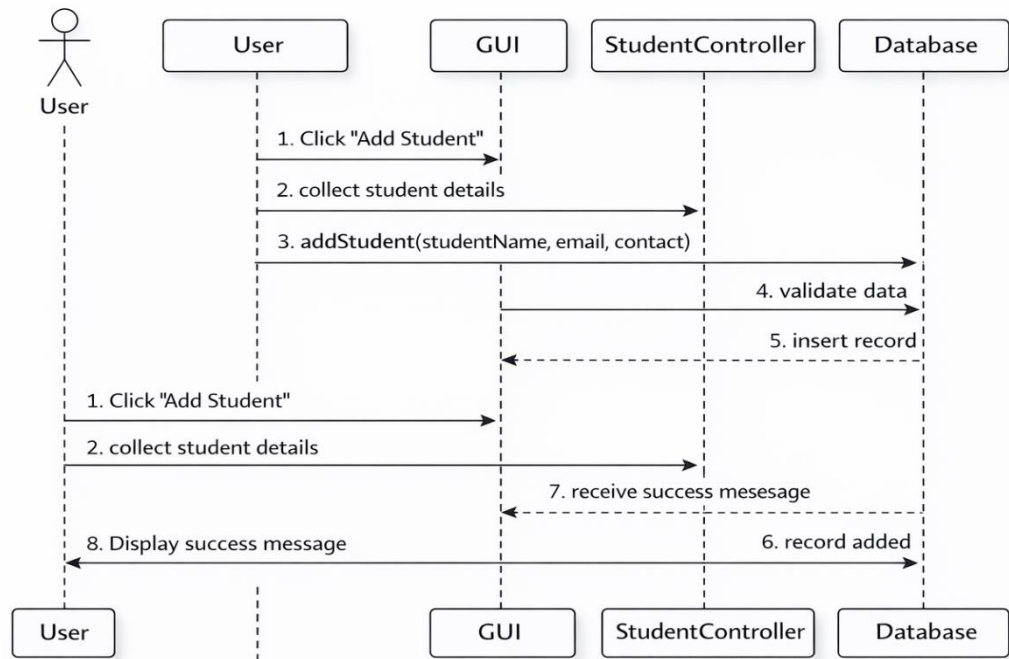


Figure 04 sequence Diagram

3.1.5 Data flow Diagram

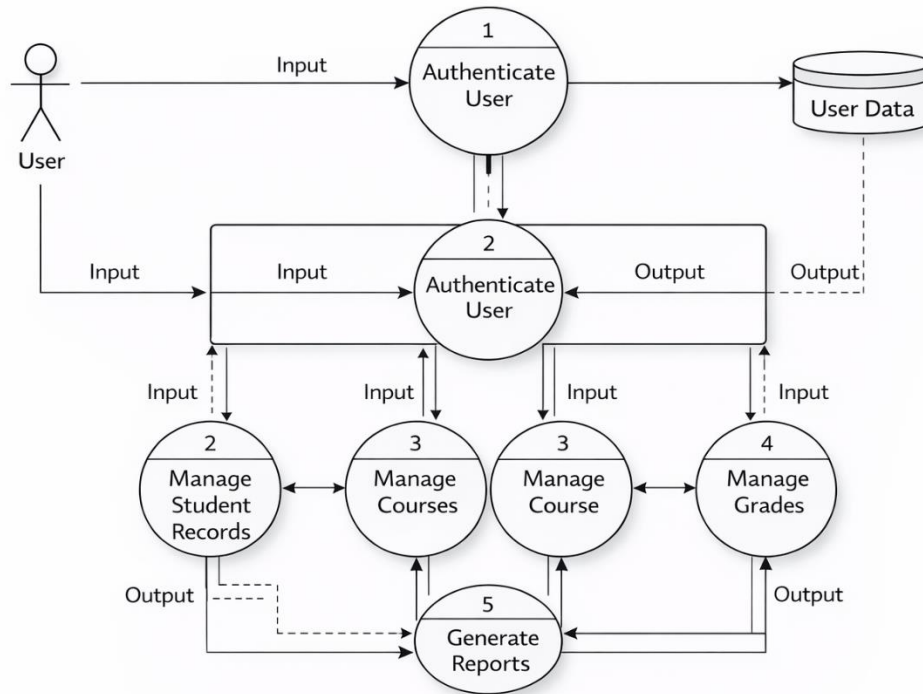


Figure 04 Data Flow Diagram

3.2 Non-Functional Requirements

3.2.1 Security

- The system shall allow only **authenticated users** to log in.
- User passwords shall be **encrypted** and not stored in plain text.
- Access rights shall be **role-based**, with Admins having higher privileges than regular users.

3.2.2 Performance

- The system shall **process record search and retrieval queries within 2 seconds** under normal load.
- The system shall support at least **50 concurrent users** without performance degradation.
- Reports shall be generated in **less than 5 seconds** for up to 10,000 records.

3.2.3 Reliability and Availability

- The system shall have an **uptime of 99%** during operational hours.
- The system shall **recover from unexpected shutdowns** without data loss.
- Daily automated database backups shall be **supported**.

3.2.4 Usability

- The system shall provide a **user-friendly GUI** with intuitive navigation.
- Users shall be able to **learn the main features within 30 minutes** of training.
- The interface shall be **consistent across all modules** (record management, report generation, etc.).

3.2.5 Maintainability

- The system code shall follow **modular design principles**, allowing easy updates.
- The system shall provide **clear documentation** for installation, configuration, and troubleshooting.

Table 1. Main System Modules

Major Modules of the System	Module Categories and Descriptions		
	<i>Module Name</i>	<i>Description</i>	<i>Subhead</i>
Functional Module	User Management	Handles user authentication	
Functional Module	Student Management	adding, updating, deleting, and viewing student records	
Functional Module	Course Management	Manages course information	
Functional Module	Grade Management	Records, updates, deletes, and views student grades for specific courses,	
Reporting Module	Report Generation	Generates academic or system reports by processing student, course, and grade data	
Data Management Module	Database Management	Stores and retrieves data for users, students, courses	

CHAPTER 4: IMPLEMENTATION

4.1 Development Environment

- **Programming Language:** Java
- **JDK Version:** JDK 24
- **Integrated Development Environment (IDE):** NetBeans
- **Database Management System:** MySQL
- **Database Connector:** MySQL Connector/J 9.4.0
- **Operating System:** Windows 10
- **Hardware Configuration:** 12 GB RAM

4.2 Implementation Details

The system was developed using **Java Swing** for designing the graphical user interface and **JDBC** for establishing connectivity between the application and the MySQL database. The application follows a **modular structure**, where each module handles specific functionalities of the system.

Student Management Module

This module handles **Create, Read, Update, and Delete (CRUD)** operations related to students.

Course Management Module

This module manages course-related information such as course ID, course name, course code, and credit value

Grade Management Module

The grade management module records and manages student grades for specific courses.

User Authentication Module

This module is responsible for secure user login and access control. It validates user credentials and restricts system access to authorized users only.

4.3 Problems Encountered

4.2.1 Database Connectivity Issues

Establishing a stable connection between the Java application and the MySQL database using JDBC was initially challenging. Errors related to incorrect connection URLs, driver configuration, and version compatibility with **MySQL Connector/J 9.4.0** were encountered. These issues were resolved by correctly configuring the JDBC driver and validating database credentials.

4.3.2 GUI Design and Layout Management

Designing a user-friendly interface using **Java Swing** required careful layout management. Maintaining consistent alignment and responsiveness across different screens was challenging. This was addressed by using appropriate layout managers and organizing GUI components logically.

4.2.3 Data Validation and Error Handling

Ensuring accurate data entry for student, course, and grade information was a challenge. Issues such as invalid input formats and empty fields were handled by implementing input validation and user-friendly error messages to prevent system crashes.

4.2.4 Managing Relationships Between Entities

Handling relationships between students, courses, and grades required careful database design. Ensuring referential integrity and avoiding duplicate or inconsistent records was challenging and was resolved through proper use of foreign keys and constraints.

4.2.5 Debugging and Testing Issues

During testing, runtime errors and logical issues were encountered, particularly when integrating different modules. These issues were resolved through systematic debugging, logging, and repeated testing of individual modules.

4.3 Solution implemented

To overcome the challenges identified during development, appropriate solutions were implemented to ensure system stability, usability, and performance.

4.3.1 Resolving Database Connectivity Issues

Database connection problems were resolved by correctly configuring the **JDBC connection URL**, database credentials, and ensuring compatibility between **MySQL**, **JDK 24**, and **MySQL Connector/J 9.4.0**. Connection testing was performed before executing database operations to ensure reliability.

4.3.2 Improving GUI Design and Layout

To enhance usability, **Java Swing layout managers** were applied consistently across all interfaces. Components were grouped logically, and consistent styling was used to improve readability and navigation within the system.

4.3.3 Implementing Data Validation and Error Handling

Input validation was implemented for all user inputs to prevent invalid data entries. Error messages were displayed using dialog boxes to guide users in correcting mistakes without interrupting system operation.

4.3.4 Managing Entity Relationships Effectively

Proper database normalization and the use of **primary and foreign key constraints** were implemented to maintain data integrity between students, courses, and grades. This ensured accurate data relationships and prevented duplication.

4.3.5 Enhancing Debugging and Testing

Systematic debugging techniques such as step-by-step execution, logging, and module-based testing were used to identify and resolve errors efficiently. Each module was tested individually before full system integration.

Table 2. Main Function types

Core Functionalities of the System	Function Types and Descriptions		
	<i>Function Type</i>	<i>Function Name</i>	<i>Description</i>
User Authentication	Security Function	User Login	Allows authorized users to access the system using valid credentials.
User Authentication	Security Function	User Logout	Allows users to securely exit the system
Student Management	Data Management Function	Add Student	Enables users to add new student records to the system.
Student Management	Data Management Function	Update Student	Allows modification of existing student information.
Student Management	Data Management Function	Delete Student	Removes student records from the system.
Student Management	Data Management Function	View Student	Displays stored student details in a structured format.
Course Management	Data Management Function	Add Course	Allows users to add new course information.
Course Management	Data Management Function	Update Course	Enables updating of course details.
Course Management	Data Management Function	Delete Course	Removes existing course records.
Course Management	Data Management Function	View Course	Displays course information stored in the system.

Core Functionalities of the System	Function Types and Descriptions		
	<i>Function Type</i>	<i>Function Name</i>	<i>Description</i>
Grade Management	Data Management Function	Add Grade	Records student grades for specific courses.
Grade Management	Data Management Function	Update Grade	Allows modification of recorded grades
Grade Management	Data Management Function	Delete Grade	Removes grade records from the system.
Grade Management	Data Management Function	View Grades	Displays grades associated with students and courses.
Reporting	Reporting Function	Generate Reports	Produces summary reports based on student, course, and grade data.

CHAPTER 5: RESULTS AND EVALUATION

5.1 System Results

The final system is fully functional and successfully performs all core tasks as intended. The implemented system demonstrates the following results:

- Successful **user authentication** and secure system access
- Efficient **student record management** (add, update, delete, view)
- Accurate **course management** and maintenance of course details
- Correct **grade recording and retrieval** for students
- Reliable **report generation** based on student, course, and grade data
- Smooth system performance with responsive GUI operations

All modules operate seamlessly and integrate correctly with the database.

5.2 Evaluation

The system was evaluated based on functionality, performance, and usability:

- The system produces **expected and accurate results** for all operations
- Data stored and retrieved from the database remains **consistent and reliable**
- Performance remains **stable with moderate data volumes**
- The graphical user interface is **clear, intuitive, and easy to learn**
- Error handling and validation reduce incorrect data entry

Overall, the system meets the objectives defined during the requirement analysis phase.

5.3 Limitations

Despite its successful implementation, the system has the following limitations:

- The system is a **desktop-based application** and does not support online or remote access
- Access is limited to **authorized users only**, with no public or student self-access
- Advanced analytics and visualization features are not included
- Scalability for very large datasets has not been tested

CHAPTER 6: FUTURE WORK

Future enhancements for the system include:

- **Developing a student login system** to allow users to:
 - View their own course enrollment and grades
 - Track academic progress and performance
 - Receive notifications for updates or alerts related to grades
- **Implementing an analytics dashboard** to provide insights such as:
 - Average grades per course
 - Student performance trends
 - Course-wise performance comparisons
- **Converting the system into a web-based application** to:
 - Allow online access for students and administrators
 - Improve accessibility from any device
 - Enhance scalability and multi-user support

These enhancements will make the system **more interactive, scalable, and user-friendly**, providing a modern solution for academic record management.

CHAPTER 7: CONCLUSION

The Student-Course-Grade Management System developed in this project successfully achieves its primary objectives of **managing student information, courses, and grades efficiently**.

The system provides a **user-friendly interface** through Java Swing and ensures reliable database interactions using JDBC and MySQL. Key functionalities such as adding, updating, viewing, and deleting student, course, and grade records are fully implemented and operate smoothly. Additionally, the system generates reports accurately and maintains data consistency across all modules.

During development, challenges such as database connectivity, GUI design, data validation, and managing entity relationships were encountered and successfully resolved, demonstrating the **robustness and maintainability** of the system.

Overall, the system provides a **secure, reliable, and efficient solution** for academic record management. It lays a strong foundation for future enhancements, including web-based access, student login features, and analytics dashboards, which will further increase scalability, accessibility, and usability.

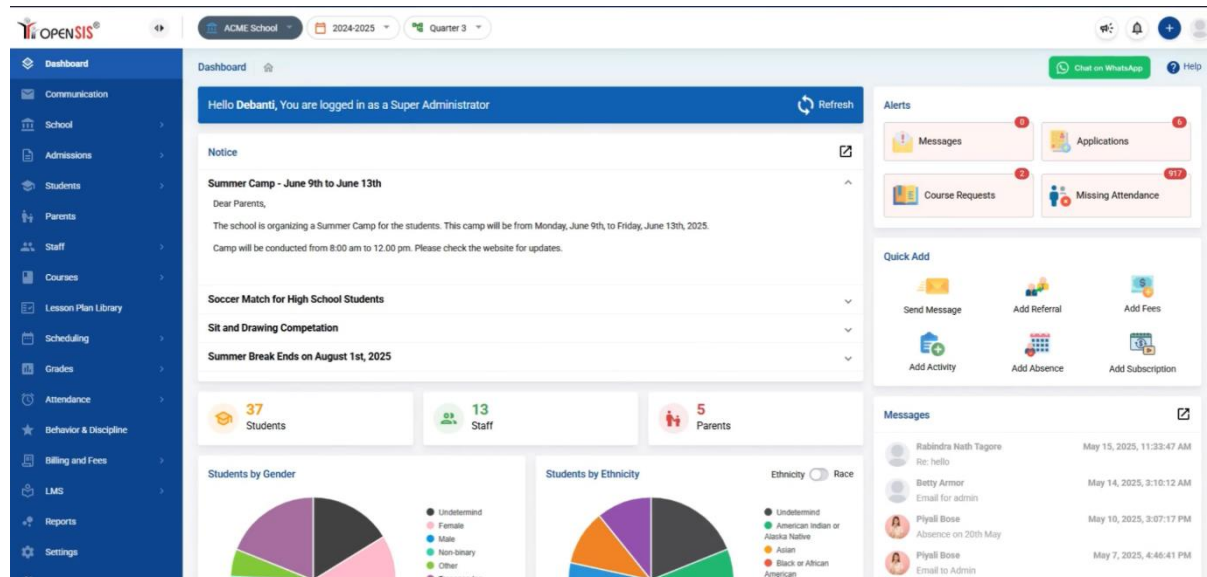
The project highlights the importance of **modular design, proper database management, and user-centered interface development** in creating practical software solutions for educational institutions.

References

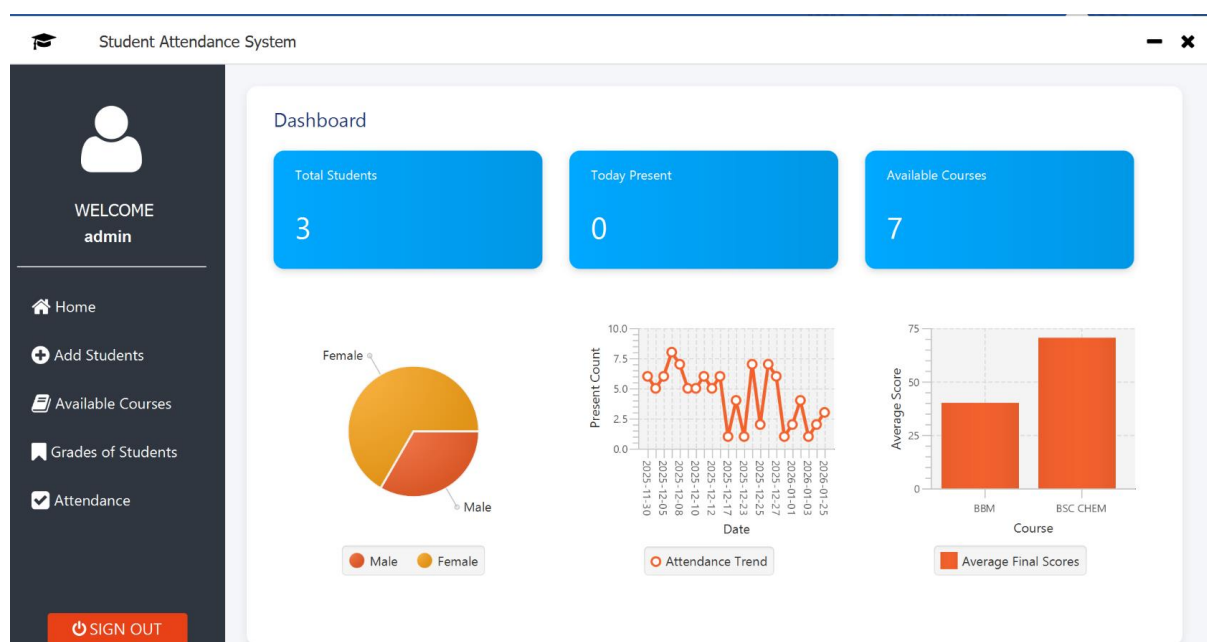
- openSIS. (2024). *openSIS: Student information system*. Open Solutions for Education. <https://www.opensis.com/>

- MySQL. (2024). *MySQL reference manual*. Oracle. <https://dev.mysql.com/doc/>
- GeeksforGeeks. (2024). *JDBC tutorial*. GeeksforGeeks. <https://www.geeksforgeeks.org/jdbc-tutorial/>

Appendices 1 : openSIS: Student information system



Appendices 2 : Homepage



Appendices 3: Add Students Form



WELCOME
admin

Home

Add Students

Available Courses

Grades of Students

Attendance

SIGN OUT

Student Attendance System

Search student...

Student No	Year	Course	First Name	Last Name	Gender	Birthday	Status
2000001	1st Year	BSC CHEM	Sanduni	Uthpala	Female	2004-01-23	Enrolled
2000002	2nd Year	BBM	Mahesh	Kumara	Male	2004-01-08	Enrolled
2000003	1st Year	BSC CHEM	Suhadi	Perera	Female	2006-01-19	Enrolled

Student No: 2000004

First Name:

Year:

Last Name:

Course:

Gender:

Birthday:

Status:

Upload Image

Clear

Delete

Add

Update

